A Menhir for Edit-Incremental Parsing: Sound, Verified Reuse and Cost-Optimal Recovery for Pratt Parsers

Jeff Pickhardt Omniscience Research Agent

Published May 26, 2026

Abstract

Production compilers ship hand-written recursive-descent parsers with Pratt expression cores. In order for tooling to keep the parse current on every edit, three approaches exist: reparse from scratch (linear in file size); run a *separate* incremental parser (tree-sitter, Lezer), with a second grammar and a tree that is not the compiler's; or use a Roslyn-like parser (Roslyn, SwiftSyntax, TypeScript) to make the hand-written parser itself incremental, via a span+lookahead reuse predicate that needs per-construct lookahead bookkeeping and is silently wrong on ~1–2 % of edits without it. We put the Roslyn approach on a firmer footing by implementing a reuse predicate read directly off the structure of Pratt parsing, sound by construction from Pratt's right-boundary property, two integer comparisons and a boundary check. This is both *generatable* and *verifiable*: a one-page operator grammar yields an edit-incremental parser plus a machine-checked soundness certificate for the predicate (bounded via Kani, unbounded via Creusot for single-byte grammars), shown across four grammars with a 400k-edit soundness oracle, and the reuse composes from expressions through a *generated* statement host into one tree. The same precedence-bounded region localizes *error recovery* with cost-optimal repair confined to the break, which we prove is the global optimum when the error is locally contained, closing a gap Diekmann named. Finally, a downstream pass memoized on node identity reuses results exactly where the parser reuses subtrees, lifting parse-level soundness one layer (identity alone is unsound under a binding change; tracking the context a value reads restores it). The result is a "Menhir for edit-incremental parsing" for the operator-expression fragment: grammar in, parser plus certificate out.

Artifact. The generator, the reference implementation, and the machine-checked certificate are available at https://github.com/pickhardt/edit-incremental-parser.

---

1. Introduction

1.1 The gap

Production compilers (rust-analyzer, TypeScript, Swift, V8) parse by hand — recursive descent with a Pratt expression core — because readability, error-reporting, and maintainability matter

more than the asymptotic generality of LR/GLR. Their tooling must reparse on every keystroke. Two families of answer dominate, each with a structural cost:

- **Reparse from scratch.** Simple and always correct, but linear in file size; a bottleneck above tens of kilobytes, and it discards AST identity across edits, defeating downstream caches.
- **A separate incremental parser** (tree-sitter, Lezer; the Wagner/Diekmann GLR lineage). These work and are widely deployed, but maintain a *second* grammar and a *second* tree that is not the compiler's. Semantic analysis then either re-runs the compiler's from-scratch parser (back to the bottleneck) or maintains a fragile cross-grammar mapping.

The Roslyn/SwiftSyntax lineage takes a middle path by making the hand-written parser incremental via a span+lookahead reuse predicate. Production implementations *are* sound, but conservatively and by hand. Roslyn's `Blender` [Lippert 2012; Roslyn 2014] extends the changed range by one token of lookahead, then reuses a green node only if it clears a battery of guards (`CanReuse`): not zero-width, missing, or incomplete; carrying no diagnostics, skipped text, or annotations; not a parser-fabricated token (including contextual keywords); and, if it spans preprocessor directives, only when the directive state is incrementally equivalent. The incomplete-node guard is a deliberately blunt catch (along the lines of "an incomplete node completes differently after a change with far look-ahead") so reuse trades rate for safety. Their soundness is based on that guard battery plus extensive testing, not a theorem. The bare span+lookahead predicate *without* the guards is silently wrong on ~1–2 % of adversarial edits. By contrast, our method replaces the guard battery with a single precedence-band check that is sound by construction. Where Roslyn rests on guards and testing, we prove soundness: a paper theorem (§3) and, for the predicate itself, a machine-checked certificate.

1.2 This paper

We give that incrementality to the hand-written **Pratt** parsers production compilers actually ship. The LR world already has it: Menhir reifies the parser state and memoizes the parse on `(state, prefix)`, reusing prior subtrees, derived by the generator [Bour 2018] (§9). But that route is unavailable to a hand-written recursive-descent frontend without rewriting it as an LR grammar, which the teams behind rust-analyzer, Roslyn, Swift, TypeScript, and V8 have declined for the error-quality and control reasons of §1.1. And a hand-written Pratt parser has *no automaton or state number* to key reuse on. Our answer to "what is the state, then?" is two integers of *precedence context* the parser already computes (§2). Three consequences follow, and they are what the LR route does not give *this* setting: the technique is a small **local retrofit** to an existing parser, not a switch to a parser generator; the reuse decision is compact enough to **machine-verify** (the LR-incremental layer offers generator-trust, and Menhir-Coq verifies *batch* LR(1), not the incremental memoization); and the same precedence structure that bounds reuse bounds **cost-optimal recovery with a composition guarantee** (Bour's recovery is production-proven but heuristic: hole-fill plus an indentation resume, with no optimality claim). We concede the converse plainly: LR/GLR are more general (ambiguous grammars and non-LL host constructs lie beyond us), and Bour's system is more mature. We serve the parsers that chose hand-written Pratt.

Concretely:

- a **precedence-bounded reuse predicate** (we call this the Top-Down Locality Principle): a cached subtree is reusable iff a two-sided precedence band contains the new `min_prec`, its text is unchanged, and its tokenization boundary is stable (§2);
- a **formalism** with mechanization-ready theorem statements and a **per-grammar certificate** of the predicate (§3);
- a **generator**: a one-page grammar yields the parser, a soundness oracle, and the certificate, demonstrated across four grammars (§4);
- **composition**: the reuse extends from expressions through a *generated* statement/declaration host — itself emitted from the spec — into one tree, reusing recursively through nested blocks (§4); and
- an **incremental-lexing layer** plus a precise account of *where* incremental lexing helps (§5); and
- **error recovery** built on the same precedence-bounded region: cost-optimal repair localized to the region around a broken edit, with a composition theorem (locally optimal = globally optimal) closing the gap Diekmann [2019] names: recovery *inside* the incremental parser (§6); and
- **identity-keyed incremental semantics**: a downstream pass memoized on node identity reuses results exactly where the parser reuses subtrees, lifting the parse-level soundness one layer (incremental memoized evaluation = from-scratch evaluation, §7). We show node identity is *necessary but not sufficient*, unsound alone under a binding change, and that tracking the context a value reads recovers soundness, invalidating only dependent uses (§7).

A through-line ties the layers together: **relative offsets.** AST nodes store widths, tokens store gap+width, the source is a rope so in every layer an edit leaves the suffix untouched and reuse/splice is cheap. Red/green trees, applied uniformly to nodes, tokens, and text.

1.3 Where this is useful

The obvious consumer is **development tooling**: a language server keeping the compiler's own AST live per keystroke, so semantic services (types, diagnostics, refactors) run on the same tree the compiler uses, with no cross-grammar mapping and a sub-millisecond reparse floor that makes otherwise-too-slow features (live whole-project analysis) feasible.

But there is a second consumer that did not exist when this problem was first studied, and that we think is now of growing importance: **AI coding agents.**

An autonomous agent editing a file does not parse once at the end. It makes a long sequence of edits and benefits from a *live structured view of the code as it works*. Maintaining an AST across the agent's edits, and letting the agent query and act on its properties between edits, changes what the agent can do:

- **Structural self-orientation.** Between edits the agent can ask the live tree where it is and what holds (am I inside a function body, a string, a comment; what is in scope at this point;

is the expression I just wrote well-formed; did my last edit break the parse), and use the answer to inform the next action, rather than re-deriving structure from raw text each time.

- **Edit at agent rates.** An agent emitting many small edits cannot afford a from-scratch reparse per edit on a large file; incremental reparse keeps the AST current cheaply, making AST-aware generation practical where batch parsing would not be.
- **A broken edit still yields a tree.** An agent mid-edit frequently holds syntactically invalid code. Error recovery (§6) keeps the live tree available, and reports *where* and at *what cost* it repaired, so the agent's structural self-orientation degrades gracefully instead of going dark; the repair cost is itself a signal of how far the last edit is from valid.
- **The reuse delta is itself a signal.** The parser already computes which subtrees changed and which were reused; an agent can use that to *scope its own re-analysis*, re-checking only what changed, mirroring how the parser scopes reuse.
- **Soundness matters more, not less, when the consumer is automated.** A human notices a corrupted AST; an autonomous loop silently reasons from it. The ~1–2 % silent-corruption rate of the naive span+lookahead predicate is a hazard for an agent. A reuse predicate that is *sound by construction* and *machine-verified* is the right substrate for automated reasoning over a continuously-edited tree.
- **One tree.** As with the IDE, the agent and the compiler's analysis should see the same tree, so the structural feedback the agent acts on is the same structure the compiler will compile.

We do not build an agent here; we note that the latency floor, the single-tree property, and the verified soundness this work provides are exactly the substrate such an agent needs, and that "a consumer making rapid sequential edits that needs a live, trustworthy structured view" now includes AI coders as a first-class case, not just human IDEs.

---

2. The Top-Down Locality Principle

Pratt's `parse_expr(min_prec)` consumes a prefix, then absorbs infix operators while their left-binding power exceeds `min_prec`. Every returned subexpression is implicitly indexed by the `min_prec` it was parsed under. Two integers characterize that precedence context and are recorded per node at parse time (no asymptotic overhead):

- **`m_spine`** — the minimum lbp of operators absorbed at the producing call's top loop (`+∞` if none). The upper bound of the reuse band.
- **`stop_lbp`** — the lbp of the token that ended the producing loop. The lower bound.

Reuse predicate (informal). A cached subtree `T` is reusable as the result of `parse_expr(M)` after an edit iff: (1) `T.stop_lbp <= M < T.m_spine` (two-sided precedence band); (2) `T`'s text is outside the edit; (3) the bytes immediately adjacent to `T`'s span are unchanged; and (4) the lbp of the next token in the new source equals `T.stop_lbp`. Cost: two integer comparisons, four byte comparisons, one lbp lookup. Soundness follows from Pratt's Property (ii): a returned subexpression is bounded on the right by a token with lbp $\leq$ the surrounding rbp. The band is exactly that property restated

as a constraint on `M`.

Conflict typology (AOPP). Each precedence conflict is *strong* (no associativity choice resolves it — dangling-else, generic `<`), *weak* (resolved by a declared associativity), or *associativity-conflict* (both directions semantically equal — `+`, `*`). The three-way dispatch on this typology is what lets associativity-conflict chains be built as balanced trees (O(log n) depth, killing the left-recursion pathology) and reused across re-association.

Relative offsets (the through-line). Nodes store byte *width*, not absolute spans (Roslyn red/green adapted to Pratt). Reuse is one `Arc::clone`; a reused subtree needs no position rewrite. The same idea returns for tokens and text in §5.

Reuse is immune to context-dependent delimiters. A `(` is a grouping `nud` or a function-call `led` depending on the token to its left, so a single edit can flip its role. This does not threaten reuse: the cache is consulted only at `parse_expr` frame boundaries, and a role-flip changes *which* frames exist, so a stale `Paren` node is never grafted where a call now sits. We confirm this with five targeted role-flip edits (including the boundary-byte-unchanged case `a + (x)` → `a  (x)`) plus the 5000-case equivalence proptest, with zero divergence. (The same context-dependence *does* bite error recovery; §6.3.)

Why each condition is necessary. Each rules out a distinct unsound reuse; dropping any one breaks on a concrete edit. *(2) Text region.* An edit overlapping `T`'s span splices stale bytes into the new tree. *(3) Tokenization boundary.* The greedy lexer is 1-byte-context-sensitive, so a neighbouring byte can re-tokenize the span: reusing cached atom `a` from `"a"` at position 0 of `"ax"` (insert `x`) would truncate the identifier; the boundary byte at old position 1 (EOF) ≠ new position 1 (`x`) rejects it. *(1) Band, upper bound* `M_new < m_spine`. A subtree `b * c` (`m_spine = 60`) reused at `M_new = 70` is wrong: fresh `parse_expr(70)` would not absorb `*` and would return `b` alone, so splicing `b * c` overruns the cursor; `70 < 60` is false. *(1) Lower bound* `stop_lbp <= M_new`. An intermediate accumulator `Binary(==, a, a)` inside `a == a && a && a` has `m_spine = 30`, `stop_lbp = 20`; the upper bound alone would accept it at `M_new = 0`, but `parse_expr(0)` keeps absorbing `&&` past it, truncating the parse; `20 <= 0` is false. *(4) Next-token lbp.* With a whitespace-skipping lexer, an edit several bytes past `T.end` can change the next *non-whitespace* token without touching the boundary byte: inserting a space inside `||` makes two skipped `|` chars, so the next real token becomes the following `(`; (1)–(3) all pass, but reuse is unsound (the `(` would be a postfix call), and only `lbp(LParen)=90 != stop_lbp=10` rejects it. All four were found necessary by property testing, not by foresight.

Balanced building. Associativity-conflict chains (`+`, `*`, `&&`, `||`) are collected flat and built into a *balanced* tree (ceil-split) rather than a left-leaning fold, giving O(log n) depth, which removes the left-recursion reparse pathology Diekmann documents (~15× worst case on long Java statement lists; our reuse speedup instead stays flat at ~3× regardless of chain length, §8). The subtlety is the `stop_lbp` of *interior* balanced nodes.

---

3. Formalism and certificate

We restate the predicate formally with a precedence-bounded region (Def 3.1), a four-part reuse predicate (Def 3.3), and a reparse functor `R: Edit → Parse(G)` with `R(e)(T)` the incremental reparse of `T` under `e`, and reduce every theorem to two named lemmas a verifier can consume.

Definition 3.1 (precedence-bounded region). For a node `T` in the previous parse tree, the *precedence-bounded region* of `T` is its byte span `[T.start, T.end]` together with the *precedence band* `[T.stop_lbp, T.m_spine)` — the interval of floors `M` for which `T` is exactly the result of `parse_expr(M)` over that span.

Lemma 3.2 (precedence-domination invariant). For `M ∈ [T.stop_lbp, T.m_spine)`: every token on `T`'s top-loop spine binds strictly tighter than `M`, and the tokens just before `T.start` and at/after `T.end` bind `<= M`. A precedence-bounded region is therefore *locally self-contained* — interior tokens bind tightly enough to be parsed by recursion into `T`'s spine, exterior tokens too loosely to participate. (The clause about exterior tokens is exactly where the context-dependent-`(` caveat forces a condition; it is sound for fixed-role operators and is the hinge of the recovery hypothesis in §6.3.)

Definition 3.3 (four-part reuse predicate). For an edit `e`, a cached subtree `T` from the previous parse, and a new floor `M`, `T` is *reusable* as the result of `parse_expr(M)` at the corresponding new position iff all four conditions hold: (1) *precedence band* — `T.stop_lbp <= M < T.m_spine`; (2) *text-region disjointness* — `T`'s old span does not overlap the edit; (3) *tokenization-boundary agreement* — the bytes immediately before `T.start` and at `T.end` are identical in old and new source; and (4) *next-token-lbp stability* — the lbp of the first token at or after `T.end` in the new source equals `T.stop_lbp`. Conditions (3) and (4) are distinct: a whitespace-skipping lexer can have its next real token changed by an edit several bytes past `T.end` without altering the immediate boundary byte (§2).

Lemma 3.4 (deterministic-Pratt-output-equality — the KEY LEMMA). If `s` and `s'` agree on `[T.start, T.end]` and on the boundary bytes just before/after it, `lbp(next_token(s', T.end)) = T.stop_lbp`, and `M ∈ [T.stop_lbp, T.m_spine)`, then `parse_expr(M)` produces *identical* subtrees on `s` and `s'`, terminating at the same byte position. By induction on the `parse_expr` call structure: the span + boundary conditions make the greedy lexer emit the same tokens; the band (via Lemma 3.2) makes every spine recursion descend in both runs; the next-token-lbp condition makes the loop terminate at the same boundary. This is the key fact under both soundness (3.5) and recovery composition (6.1).

The four-part predicate's conditions are exactly Lemma 3.4's hypotheses, so the theorems follow (each stated for mechanization):

Theorem 3.5 (soundness). `R(e)(T) = batch_parse(apply(e, text(T)))` — the incremental reparse equals a fresh parse of the edited source.

Theorem 3.6 (reuse-optimality). The predicate reuses maximally on the operator-expression fragment: any sound reuse algorithm reuses no subtree the predicate rejects.

Theorem 3.7 (escalation completeness). When reuse fails at the innermost node, escalation terminates in O(depth) and equals a fresh parse.

The same two lemmas carry the two novel results stated and proved later: **recovery composition** (Theorem 6.1, §6) — locally-optimal repair = globally-optimal when the region is the *sole error locus*, the gap Diekmann names; an earlier formulation assumed only *boundary* well-formedness, which property testing falsified (§6.3) — and **semantic soundness of identity-keyed reuse** (Theorem 7.1, §7).

The certificate is what a generator emits. This important predicate is small enough to verify automatically and to *regenerate per grammar*: bounded and bit-precise via Kani/CBMC (all grammars), and unbounded over `Seq<u8>` via Creusot/Why3/SMT (single-byte grammars, matching the calculator setting where conditions 3 and 4 coincide).

---

4. Generation and composition

4.1 A generator

`incremental-pratt-gen` reads a one-page grammar spec (operators with precedence, fixity, associativity, conflict class) and emits a standalone crate: a from-scratch parser, the edit-incremental parser, a soundness oracle, and the certificate. The expression *engine* — the predicate, the Arc/width AST, the balanced builder — is shared runtime, **byte-identical across grammars**; only the lexer, operator tables, and statement-grammar table are generated. We demonstrate four grammars from one codegen path: `calc` (the mechanized instance), `c_expr` (multi-byte operators), `assoc_stress` (four associativity-conflict operators, stressing the balanced builder where the historical interior-`stop_lbp` soundness bug lived), and `stmt_lang` (a custom statement front-end — `let`/`print`/`def`/blocks/expression statements — over the same engine, evidence the statement grammar is a genuine spec parameter, not a hand-written fixture). Each passes the oracle and the Kani certificate.

The generatability claim is scoped deliberately. In the *LR* world, Menhir already derives sound incremental subtree reuse from the automaton. For instance, Bour [2018] memoizes the parse on `(state, prefix)` and short-circuits to a prior subtree on a known pair, with the generator's own correctness guarantees (§9). What is new here is the *top-down* analog: a generator for the hand-written **Pratt** parsers production compilers actually ship, where there is no LR automaton or state number to key reuse on, and the reuse key is instead the precedence band read off Pratt's structure (§2). We claim generatable sound reuse for that setting, not in general.

4.2 Composition: a statement/declaration host

We demonstrate the single-tree thesis scales past expressions. A statement/declaration layer wraps the generated expression core, and the whole program tree is reparsed with **two granularities of reuse into one tree**: statement-level **reparseable-element** reuse (the mechanism

IntelliJ/Roslyn/SwiftSyntax use — reuse the statements an edit doesn't touch) composed with expression-level precedence-bounded reuse inside the edited statement. The statement grammar is itself **generated** from the spec's `[host]` section (a default `let … ; | expr ; | { … }` grammar when none is given), so the host is grammar-driven rather than a hand-written fixture; the interpreter and incremental-reuse engine are byte-identical across grammars. Reuse **recurses into nested blocks**: an edit deep inside nested `{ … }` reparses only the innermost enclosing production whose boundary token is unchanged, reusing every sibling subtree wholesale and escalating one level at a time, bottoming out at a sound full reparse. A one-byte edit in an expression nested two blocks deep reparses only the innermost enclosing statement, with no reparse above it: every untouched statement at every enclosing level is reused wholesale, and the expression subtrees inside the edited one are reused via the band (`host_recursive_reuse_demo`). The incremental program reparse equals a fresh parse over 50k random edits per grammar, across all four grammars (three on the default host, `stmt_lang` on a custom one) and exercising nested blocks and multi-edit chains (`host_incremental_matches_fresh`, `host_chain_matches_fresh`); and for each grammar a synthesized program covering every statement alternative round-trips through an edit (`host_sample_roundtrip`).

One scoping choice is deliberate and is exactly the asymmetry §2 turns on. Statement reuse here is sound by *guard plus oracle* — three guards re-validated against the source (the edit lies in a single child's span; a reparsed subtree consumes exactly its delta-adjusted span, the one-token-lookahead guard made exact; an expression-leaf replacement introduces no host-token bytes) backstopped by the equivalence oracle — **not** by a derived certificate the way the expression predicate is (§3). It *could* be: the host is a *generated* LL recursive-descent parser, so its sound reuse key is readable off its own structure — the production context plus the bounded prediction (FIRST/FOLLOW) lookahead each decision consults — the discrete, set-valued analog of the numeric precedence band (statement grammars are not precedence-driven, so the analog is an equivalence class of contexts, not an interval; bounded for LL(k), though §9's non-LL(k) host constructs need unbounded lookahead). The argument is the LL counterpart of Lemma 3.4 and needs no LR automaton or parser-generator rewrite. We leave it out for two reasons, neither an impossibility. First, that key is the standard incremental-LL/LR reuse idea (Wagner; tree-sitter; Bour/Menhir; with verified *batch* parsers in Menhir-Coq/CoStar), so building it applies known ideas and uses none of this paper's one new mechanism, the band. Second, *certifying* it is a machine-checked proof over the recursive-descent host interpreter, which hits the same parser-symbolic-execution wall (§3, §6.6) that keeps the parser and recovery search out of our certificate. So we certify the band by construction and leave the statement layer on the reparseable-element mechanism, oracle-validated. (Strong-conflict and recovery in the host remain out of scope.)

---

5. Incremental lexing, and where it helps

We add four lexing layers and, more usefully, a precise account of which consumer each serves.

- **Demand-driven lexing.** The parser pulls tokens on demand and jumps the lexer past reused subtrees, so it lexes only what it visits (~0.2 % of the file's tokens for a local edit on

a reuse-rich source).

- **Persistent token store.** An implicit treap with relative offsets; O(log n) splice and byte→index lookup.
- **Relex-to-resynchronization.** Maintains the store in O(edit + resync), proven equal to a full re-tokenize, with a constant relex window as the file grows (worst case O(n) for an edit whose lexical effect runs to EOF).
- **Incremental document.** Source text in a rope + the token stream, both updated per edit with no full re-copy or re-lex; a `ByteSource` trait lets the same generated lexer read a slice or the rope.

The taxonomy (the point). For a cheap lexer, demand-driven lexing already captures the parser-side win, reading a token from an O(log n) store is no cheaper than re-lexing it at O(1), and a low-reuse edit makes the parser traverse the tokens regardless. The persistent store + relex earn their keep for a *different* consumer: the whole token stream (syntax highlighting, semantic tokens) that an editor, or an AI agent's structural self-orientation, needs current on every keystroke independent of parser reuse, and for expensive lexers where avoiding a re-lex saves real work. Stating this split and proving `relex ≡ full-tokenize` is an important result.

---

6. Error recovery via precedence-bounded regions

We now build the recovery-composition theorem from §3. The same precedence-bounded region that bounds *reuse* also bounds *repair*: when an edit makes the parse fail, the fix is confined to the region around the break, everything outside is untouched, and (under a condition we make precise below) that local repair is globally optimal. Recovery and incrementality have been *paired* in production: Merlin runs `(state, prefix)`-memoized reuse alongside hole-filling recovery (§9), but as separate mechanisms with no guarantee relating them. What is new here is the *composition*: recovery confined to the incremental reuse loop with a local = global cost-optimality guarantee, the gap Diekmann [2019, §3.1] names as unfilled.

6.1 Cost-optimal repair, localized

A *repair* is a minimum-cost sequence of token insert / delete / substitute operations that makes a broken stream parse, valued in the min-plus tropical semiring `(Cost ∪ {∞}, min, +)`. Only the monoid structure (additivity of non-negative costs) does real work in what follows; we keep the semiring vocabulary where it earns its place (the composition statement, the CPCT+ connection) and say so where it does not.

The region localizes the search. An error inside a precedence-bounded region is repaired by searching only within that region; the rest of the input is neither re-searched nor re-parsed. Localization is by *precedence*, not merely by brackets: an error deep in a flat operator chain `a * + b + c + …` is repaired in a window bounded by the precedence valley around the break, constant in the length of the surrounding chain.

6.2 The composition theorem

> **Theorem 6.1** (locally optimal = globally optimal for a sole-error region). Let `R` be a precedence-bounded region and `e` an edit in `R`'s interior. Suppose the **sole-error condition**: replacing `R`'s interior with a placeholder operand yields a token stream that parses ("`R`'s skeleton parses"). Then the minimum-cost repair restricted to `R` equals the minimum-cost repair over the whole input.

Setup. Repair cost lives in the min-plus monoid: a repair is a sequence of token insert/delete/substitute operations, its cost is the sum of the operation costs (a top element marks an impossible repair), and it is *complete* for an input when the repaired stream parses. Write `local` for the minimum-cost complete repair whose operations all lie within `R`, and `global` for the minimum-cost complete repair over the whole input. The recovery scheme of §6.1 resets its cost accumulator at each precedence-bounded boundary, so no repair propagates across one.

Proof. Any repair confined to `R` is in particular a repair of the whole input, so `cost(global) <= cost(local)`. Suppose for contradiction the inequality is strict. Then `global` includes at least one operation outside `R` — otherwise it would itself be a complete repair confined to `R` cheaper than `local`, contradicting `local`'s optimality. Partition `global` into its operations inside `R` (call them `inside`) and the rest (`outside`); costs are non-negative and additive over the disjoint sets, so `cost(global) = cost(inside) + cost(outside) >= cost(inside)`.

The load-bearing step is that **`inside` alone is a complete repair of `R`**. The precedence-domination invariant (Lemma 3.2) makes a precedence-bounded region locally self-contained — its parseability depends only on its own interior and boundary tokens, not on anything outside — but, as Lemma 3.2's exterior-token clause flags, only for *fixed-role* tokens; the one exception is a context-dependent delimiter whose role is set from outside `R` (the `(` of §6.3). The sole-error hypothesis is exactly what rules that exception out: if `R`'s skeleton parses, no outside-driven reinterpretation of `R` is in play, so Lemma 3.2 applies and `R` is self-contained. Hence no operation in `outside` can contribute to making `R` parse — the operations in `inside` must already repair `R`'s interior to a self-contained operand, which the surround accepts because the skeleton parses. So `inside` makes the whole input parse: it is a complete repair confined to `R` with `cost(inside) <= cost(global) < cost(local)`, contradicting `local`'s optimality. Therefore `cost(local) = cost(global)`. ■

The proof uses only additivity of cost over disjoint operations and optimality-as-minimum — the monoid's `+` and `min`, no distributivity or deeper semiring identity. The substantive work is done by the sole-error hypothesis and Lemma 3.2, not the algebra. The next subsection is why the hypothesis must be *sole-error* and not merely boundary well-formedness.

6.3 What building it taught us: the clean hypothesis was wrong

We first stated Theorem 6.1 with a weaker hypothesis, that `R`'s *boundary tokens* are well-formed, and a proof that invoked the precedence-domination invariant to claim "`R`'s parseability is independent of what happens outside `R`." Property testing (2000 random valid-expression × corrupting-edit cases) falsified it. The minimal counterexample is `(()`:

- Let `R` be the empty interior of the inner parentheses; its boundary `)` is present and well-formed.
- Local repair must fill the empty parens — cost 2. But the global optimum *substitutes the outer `(`* for an operand, turning the empty `()` into a valid empty **call** — cost 1, editing nothing inside `R`.

The cause is a context-dependent delimiter: `(` dispatches as a grouping `nud` or a call `led` depending on the token to its *left*, which lies outside `R`. So an edit outside `R` can change `R`'s meaning: the "independent of outside" claim is false. It is the classic function-call wrinkle of operator-precedence grammars. The **sole-error / skeleton** hypothesis is exactly what rules this out: `(())`'s skeleton `((0)` does not parse, so the region escalates rather than repairing locally. Two earlier failure modes fall under the same corrected condition: an error inside an unclosed group (`a[`), and a stray unbalanced closer as the boundary (`(a) + )`).

This is a correction a paper-only proof would have shipped; the implementation caught it. We report it because it sharpens the theorem, and because the corrected hypothesis is directly *checkable*: the parser decides a region's local reusability by literally testing whether its skeleton parses.

6.4 Soundness reduces to one condition

The sole-error check is, by itself, sufficient for soundness. We verified this by *relaxing* the region-proposal machinery (the bracket-scope and precedence-valley heuristics that propose a tight window) and confirming the corpus and the 2000-case proptest still pass with only the skeleton check active. So the proof needs one hypothesis, not three; the region machinery is an *optimizer* (it keeps the window tight, reports honest escalation, and avoids search calls on windows the skeleton check would reject), not a soundness condition. When the sole-error condition fails, recovery escalates to the enclosing region and ultimately to a whole-input repair; escalation terminates by the escalation-completeness theorem of §3.

6.5 Recovery inside the incremental parser

Recovery composes with reuse rather than replacing it. A clean edit takes the fast precedence-bounded *reuse* path unchanged; an edit that introduces a syntax error falls through to localized repair confined to the region around the break, with cached subtrees outside the region intact. The result is an incremental reparse that never fails: it returns a tree, the repairs that produced it, and the region they were confined to. *(Reference implementation: the repaired tree is rebuilt by the search rather than sharing `Arc` subtrees with the cache - the search is region-local, but Arc-reuse during recovery is future work. Clean edits retain full Arc-reuse.)*

6.6 Relation to CPCT+, and scope

CPCT+ [Diekmann and Tratt 2020] performs minimum-cost LR repair within Wagner's stack isolation; it is the LR-stack-isolation instantiation of the same composition argument (precedence-bounded region $\mapsto$ isolation region; sole-error $\mapsto$ unchanged isolation lookahead). We do not improve on CPCT+; the uniformity is the point.

Scope (stated plainly). Recovery is built and verified in the reference implementation (`reference-impl` / `verification`) on the `calc` grammar — the same instance the generator's certificate mechanizes. It is *not yet emitted by the generator*; generating recovery across grammars is narrowed future work. The cost monoid Theorem 6.1 turns on is Kani-verified (§8); the repair *search* and the parser are not (the parser-symbolic-execution wall of §3), and rest on the paper proof plus the 2000-case property test.

---

7. Identity-keyed incremental semantics

In §1.3, we promised a consumer (IDE or agent) that keeps the tree live and *scopes its own re-analysis to what changed*. This section makes that concrete and states its soundness. The claim is that the parse-level soundness of §3 lifts one level: a downstream semantic pass, memoized on node identity, computed over the *reused* tree, equals the same pass computed from scratch over a fresh parse. Parsing is rarely the bottleneck; semantic analysis is. So this is where the latency floor actually pays off.

7.1 Identity as the reuse key

Reuse is `Arc::clone`, so a reused subtree is the *same allocation* across an edit; in the reference implementation a stable per-node id (the `node_id` build, zero cost when off) is preserved across the `make_mut` that restamps boundary metadata, giving each subtree an identity that survives reuse. A semantic pass memoized on that id reuses a subtree's result in O(1), without revisiting its interior, exactly when the parser reused the subtree, and recomputes exactly what the parser rebuilt. For a context-free interpretation (an integer `eval` over the calculator grammar, `reference-impl/src/semantics.rs`) the per-edit recompute therefore tracks the parser's rebuild set, not the file: editing one argument of a many-argument call reuses every sibling argument as a single memo hit (`recompute_is_local_when_siblings_are_reusable`).

A subtlety we surface rather than hide: on a flat associativity-conflict chain, the default builder rebuilds the whole O(n) balanced interior per edit (§5.1 of the companion analysis), so identity-keyed eval recomputes O(n) — faithfully tracking parse reuse. The optional `chain_splice` path (in-place, and operand insert/delete via splitting the chain into O(log n) units and rebuilding a balanced spine over them) reduces both the parse rebuild and the eval recompute to O(log n) *per edit* — measured ~204 → $ $48 recomputed for an n=200 chain. It is sound precisely because the ≈-quotient lets the splice emit *any* balanced regrouping of the same operands. Keeping depth bounded across *unbounded* edits required a real self-balancing structure: naively rebuilding a spine over the units (count- or width-split) lets imbalance accumulate inside the reused units, so depth grows ~log(edits). The insert/delete splice therefore *merges* the unit spines with a leaf-based weight-balanced (BB[α]) `join` (rotations), maintaining the weight-balance invariant on the whole chain; depth then plateaus at the weight-balanced constant ($\sim 2.4 \cdot \log_2 n$) and is stable across edits rather than creeping (measured: stable at ~18 for a 64-operand chain over 300 edits, with the per-node weight-balance invariant asserted on every edit). The in-place path-copy splice is exactly depth-

preserving. Soundness across both paths is property-tested at 50k incremental-vs-fresh cases (≈, the chain may regroup).

7.2 Identity is necessary but not sufficient: context must be tracked

Node identity certifies that *syntax* is unchanged. It does **not** certify that *meaning* is unchanged: a value that reads a binding (name resolution, a type, a constant's value) can change while the syntax node is byte-for-byte identical. Memoizing on identity alone is therefore unsound for context-dependent semantics. We make this concrete with a variable environment `Γ: name → value` interpreting identifiers (`reference-impl/src/semantics_ctx.rs`):

- **Negative result.** A NodeId-only memo (`NaiveNodeIdEvaluator`) primed on `Γ = {a:5}` returns the stale value `6` for `a + 1` when `Γ` later becomes `{a:10}`, where the truth is `11` — the tree, and its ids, are identical (`naive_nodeid_memo_is_unsound_under_binding_change`). Identity alone is unsound under a binding change.
- **Positive result.** Track, per node, the bindings the value read, and reuse a memo entry only when the node identity *and* every read binding are unchanged (`ContextEvaluator`). A binding edit then invalidates exactly the uses that read it — even though those use-nodes are syntactically unchanged and keep their ids — while subtrees that did not read it are reused (`binding_change_invalidates_only_dependent_uses`: changing `a` reuses the `b`-only subtree with zero source edits). A source edit reuses any subtree whose identity and read-context are both unchanged. Soundness is property- tested over 3000 random expression × two-environment cases (`ctx_eval_sound_under_binding_change`).

This is the self-adjusting-computation / salsa / Adapton discipline [Acar 2009; Hammer et al. 2014], specialized to parse trees: identity is the *syntactic* leaf of the dependency graph; context-dependent facts are tracked edges. The contribution here is not that discipline — it is that the parse layer beneath it is *sound and identity-stable by construction*, which is the property a downstream incremental analysis must be able to assume and, on a heuristic reuse layer, cannot.

Memo memory is bounded by a threshold-triggered reachability GC (retain only entries reachable from the live tree, reset the threshold to twice the live size) — O(1) per edit, perf-neutral, so the memo tracks the live tree rather than leaking across a long edit session.

7.3 The lifted soundness statement

Let a semantic interpretation `S` be a pure function of a node and the context inputs it reads, written `S(n, Γ)`, and ≈-invariant (it respects the associativity-conflict firm-semantics convention, so a regrouped chain denotes the same value; ordinary value/type interpretations are). Let `M` be the memoizing evaluator that reuses a cached `S(n, Γ_old)` for node `n` iff `n`'s identity is unchanged *and* every context input `n` previously read is unchanged in `Γ`, recomputing otherwise. Write `R(e)(T)` for the incremental reparse of tree `T` under edit `e` (§3) and `batch` for a fresh parse.

> **Theorem 7.1 (semantic soundness of identity-keyed reuse).** For every tree `T`, edit `e`, and context `Γ`,

`M( R(e)(T), Γ ) = S( batch(apply(e, text(T))), Γ ).`

Proof sketch. By parse soundness, `R(e)(T) ≈ batch(apply(e, text(T)))`, so the two trees agree up to associativity-conflict regrouping; `S` is ≈-invariant, so it is indifferent to that. It remains to show `M` over `R(e)(T)` equals `S` over `R(e)(T)`. By induction on the tree: at each node `M` either recomputes (equal to `S` by definition) or reuses, which it does only when the node's identity is unchanged (so by parse soundness the subtree is the one a fresh parse produces) and every context input it read is unchanged (so `S` returns the same value). Hence each reused result equals the fresh one. ■

This is the parse-level soundness lifted through one semantic layer; it is what licenses an agent or IDE to scope re-analysis to the reuse delta and trust the result. **Scope, stated plainly.** This is a paper proof plus a property- tested proof-of-concept (`reference-impl`: context-free `semantics.rs`, context- dependent `semantics_ctx.rs`; the soundness proptests above), not a mechanized theorem. The important hypothesis is ≈-invariance of `S`: it holds for value/type interpretations and any pass respecting the firm-semantics convention; an interpretation sensitive to associativity *grouping* would require strict-shape reuse, which the in-place splice (not the regrouping general splice) provides. Mechanizing this lifted statement, the natural successor to the predicate certificate, is the principal open item.

---

8. Evaluation

Four grammars, one generator. *(Apple M-series; ratios are the reportable figure.)*

- **Grammar-parametric generation:** four crates, runtime byte-identical.
- **Soundness:** ~400k incremental-vs-fresh equivalence cases, zero divergence (`random_sources`, `structured_sources`); plus the token-store, relex, document, and host oracles — 7 oracle tests + 3 lib tests per grammar.
- **Speed:** parser+lexer reparse 9–16× faster than from-scratch on nested sources at 32 KB, reuse ~99.9 %, `lexed %` ~0.2 %.
- **Long associative chains (the left-recursion pathology):** balanced building keeps the reuse speedup *flat in chain length* — a steady ~3× from 100 to 2500 operands (`reference-impl`) — rather than the reparse blow-up a left-leaning fold suffers (the ~15× worst case Diekmann documents).
- **Incremental lexing:** `relex ≡ full-tokenize` over 50k × 4 edits (incl. multi-byte); relex window constant as the file grows 16×.
- **Composition:** the host program reparse equals fresh over 50k × 4 edits.
- **Recovery (Theorem 6.1):** 2000 random valid-expression × corrupting-edit cases — local repair cost = global oracle cost on every case; the three theorem-hypothesis counterexamples (`a[`, `(a) + )`, `(()`) surfaced here. Recovery inside the incremental parser is validated separately: clean edits reuse with no repairs, breaking edits recover cost-optimally, and the local case keeps a constant-size repair window.
- **Identity-keyed semantics (§7):** incremental memoized evaluation = fresh over 3000

context-free edits and 3000 context-dependent expression × two-environment cases (zero divergence); the NodeId-only memo is exhibited diverging under a binding change (the negative result); a binding change reuses non-dependent subtrees with zero edits. With the `chain_splice` path, flat-chain recompute drops from ~n to O(log n) (n=200: ~204 → $ $48), insert/delete included — sound under ≈ across 50k equivalence + 3000 chain insert/delete cases. The insert/delete path uses a weight-balanced `join`, so chain depth is stable across edits (a 64-operand chain holds at ~18 over 300 edits) with the per-node weight-balance invariant asserted on every edit.

- **Certificate:** *reuse predicate* — Kani 2 harnesses/grammar (~109–112 checks, all SUCCESSFUL), Creusot `Proved` on the single-byte grammars; *recovery cost monoid* — 6 Kani harnesses (associativity; the important monotonicity inequality `add(inside, outside) >= inside`; overflow-safety; the `min` semilattice; distributivity; cost-consistency), all SUCCESSFUL.
- **Reproducibility:** `make smoke | all | verify | verify-creusot`; a claims↔evidence table; a container for the non-Creusot tiers. Artifact: https://github.com/pickhardt/edit-incremental-parser.

Per-harness certificate detail. The certificate is two layers: bounded (Kani/CBMC) for every grammar, and unbounded (Creusot/Why3/SMT) for the single-byte grammars where predicate conditions (3) and (4) coincide. What each harness verifies:

Harness	Property verified	Scope	Result
predicate (well-formedness · post-condition · negative)	`reuse` ⟺ the four conditions of Def 3.3; no panic / UB	symbolic source up to 32 B; 2 harnesses/grammar	✓ (~109–112 checks/grammar)
predicate, unbounded	same bi-implication over `Seq<u8>` of any length	single-byte grammars (Creusot)	Proved
recovery `add` associative	min-plus monoid associativity	`a,b,c <= ∞`	✓
recovery `add` commutative + identity `0`	monoid commutativity / unit	`a,b <= ∞`	✓
recovery `add` monotone + bounded	`add(a,b) >= a` (the Theorem 6.1 step); `<= ∞`	`a,b <= ∞`	✓
recovery `min` semilattice	assoc / comm / idempotent; identity `∞`	`Cost`	✓
recovery `+` over `min`	semiring distributivity	`a,b,c <= ∞`	✓
recovery `total`	`n` unit repairs cost `n`; stays `<= ∞`	≤ 3 repairs	✓

Not certified, and stated as such (§6.6): the repair *search* and the parser itself, both beyond bounded model checking (the parser-symbolic-execution wall). The composition theorem rests on the paper proof plus the 2000-case proptest.

---

9. Related work

- **tree-sitter / GLR** (Wagner, Diekmann): general-purpose incremental parsing, and the honest ceiling on our generality. Three things lie *entirely outside* the operator-expression fragment we claim: genuinely **ambiguous** grammars (GLR parses a forest; a deterministic parser cannot); **non-LL(k)** host constructs needing unbounded lookahead (C++ `T(x);`, generic `<`, C cast-vs-paren); and **strong-conflict** constructs (dangling-else, ASI) we *escalate* on rather than reuse across. Formally, operator-precedence languages sit strictly inside the deterministic CFLs LR(1) recognizes, themselves inside the ambiguous grammars GLR handles — so we are no replacement for tree-sitter, and beat it only on the left-recursion pathology. Within expressions, though, Pratt wants for nothing: the nud/led split even parses the function-call `(` that Floyd's formal operator-precedence grammars choke on (the same context-dependence that bit recovery). And on *error recovery*, §6 closes the composition gap Diekmann [2019, §3.1] leaves unaddressed.
- **Roslyn / SwiftSyntax** span+lookahead: the production lineage our predicate improves on (sound-by-construction vs. per-construct lookahead).
- **Menhir-Coq / CompCert / CoStar / Verbatim**: verified *batch* parser generation; our certificate is the edit-incremental analog, covering the predicate.
- **SiDECAR / Bianculli et al. [2013]** (the Politecnico di Milano operator-precedence-languages program): the closest academic prior art for *operator-precedence locality as a reuse criterion*. Their Proposition 1 establishes, bottom-up over operator-precedence grammars, that a local subtree replacement is sound when the surrounding derivation factors through it, the shift-reduce analog of our precedence band. We are the top-down (Pratt) realization, stated in the parser's native `m_spine`/`stop_lbp` variables with the four-part predicate and its mechanization.
- **Cost-based recovery** (Burke-Fisher [1987]; Corchuelo et al. [2002]; CPCT+ [Diekmann and Tratt 2020]): mature for *batch* / push-incremental LR, repaired per error episode. We combine cost-based repair with edit-incremental reuse, which none of these do; CPCT+ is the LR-stack-isolation instantiation of our composition theorem.
- **Merlin / Menhir** (Bour et al. [2018]): the closest *practitioner* prior art, on both our axes. Bour memoizes the parse on `(state, prefix)`, short-circuiting to a prior subtree on a known pair — the LR-automaton analog of our precedence band, *derived by the generator*. So the LR route already has sound, generatable incremental reuse; ours is the top-down/Pratt counterpart for the hand-written parsers production compilers ship, where there is no automaton or state to key on. The two also differ on recovery: Merlin's is *completion plus resume* (grammar-author "dummy" constructors, then an indentation heuristic) — production-proven but with no cost model or optimality claim, and paired with incrementality as a *separate* mechanism.

Ours is cost-optimal, and the gap we fill is precisely their *composition* under a local = global guarantee. Bour argues for the generator route outright ("if a reasonable LR grammar exists, use a parser generator without hesitation"); we serve the teams who chose hand-written Pratt anyway.

- **Lawlor's `ciaran16/incremental-parser`**: a declaration-driven Pratt-incremental library that prefigured the generated form; our addition is the soundness account, the verification, and the generator framing.

---

10. Future work

Of the directions this opens, four are the ones we would prioritize, in rough order:

- **Retrofitting a production hand-written frontend.** The predicate is a *local* change to an existing Pratt parser, not a switch to a generator (§1.2), so the natural validation is to graft it into a real frontend (rust-analyzer, TypeScript, Swift, V8) and replay recorded edit traces, comparing reuse rate and reparse latency against that project's current incremental machinery. The open questions are how much per-node bookkeeping a production AST tolerates, how the precedence band interacts with these languages' non-LL(k) host constructs, and whether the measured silent-corruption gap over a naive predicate matches our adversarial 1–2 %. This is the external-validity test of the paper's central claim.
- **Mechanizing the lifted semantic-soundness theorem (§7).** Currently a paper proof plus a property-tested proof-of-concept; this is the natural successor to the predicate certificate.
- **Incremental name resolution, types, and borrow checking on the identity-keyed substrate.** §7 lifts parse soundness through one *context-free* semantic layer and shows the discipline (track what each node reads, invalidate only dependent uses) for a context-dependent one. Extending it to real analyses — scope/name resolution, type inference, ownership — is the path to the language server of §1.3 that re-runs only what an edit can affect. The research content is identifying the right context edges per analysis (the binding edges of §7.2 generalized to import graphs, trait resolution, region constraints) so the salsa/Adapton-style memo stays sound across edits while resting on a parse layer that is sound and identity-stable *by construction* rather than heuristically.
- **AI coding agents on a live, sound tree.** §1.3 argues that an autonomous editing agent needs precisely what this work provides: a structured view kept current *between* edits, re-analysis scoped to the reuse delta, and a tree that survives a broken mid-edit state. Soundness should matter more for an agent than for a human, since the agent might silently reason off a corrupted tree. What remains is to build it and measure the payoff: whether a live, sound tree improves an agent's edit quality or only its latency. We provide the substrate, not the agent.

---

11. Conclusion

The reuse key a hand-written Pratt parser needs was already there: the two-integer precedence band, `m_spine` and `stop_lbp`, the parser computes as it runs. Reading it off Pratt's structure turns subtree reuse from a battery of hand-tuned guards (Roslyn's route, silently wrong on ~1–2 % of edits without them) into a four-part predicate that is sound by construction and small enough to machine-check. A one-page grammar spec yields the parser, a soundness oracle, and a per-grammar certificate in two tiers (bounded via Kani, unbounded via Creusot) demonstrated across four grammars: grammar in, parser plus certificate out. For a compiler you control with a Pratt expression core, this is the edit-incremental, top-down analog of verified batch parser generation — the counterpart, for parsers with no LR automaton to key reuse on, of what Menhir gives the LR world.

The same precedence-bounded region does more than bound reuse. It localizes *error recovery* — cost-optimal repair confined to the break, globally optimal when the region is the sole error locus, the composition Diekmann named as missing — and, lifted through node identity, it bounds *incremental semantics*: a downstream pass that recomputes exactly what the parser rebuilt, sound across a binding change once each value's read-context is tracked. One structure, read three ways: which subtrees to reuse, where a repair is confined, and what a re-analysis must revisit. We claim this for the operator-expression fragment, not as a replacement for LR/GLR generality, and we leave the repair search and the parser itself outside the certificate.

What the work delivers is a sound, single, identity-stable tree maintained at edit rates. As code editing shifts from humans in an IDE to agents in a loop, that could become a useful substrate for automated reasoning over continuously-edited code.

References

Acar, Umut A. *Self-Adjusting Computation: An Overview.* Foundations and Trends in Programming Languages. Now Publishers, 2009.

Bianculli, Domenico, Antonio Filieri, Carlo Ghezzi, and Dino Mandrioli. *A Syntactic-Semantic Approach to Incremental Verification.* 2013. arXiv:1304.8034. (Published 2015 as "Syntactic-semantic incrementality for agile verification," *Science of Computer Programming* 97:47–54.)

Bour, Frédéric, Thomas Refis, and Gabriel Scherer. *Merlin: A Language Server for OCaml (Experience Report). Proceedings of the ACM on Programming Languages* 2(ICFP):103:1–103:15, 2018. doi:10.1145/3236798.

Brunsfeld, Max. *tree-sitter: An Incremental Parsing System for Programming Tools.* 2018. https://tree-sitter.github.io/tree-sitter/.

Burke, Michael G., and Gerald A. Fisher. *A Practical Method for LR and LL Syntactic Error Diagnosis and Recovery. ACM Transactions on Programming Languages and Systems* 9(2):164–197, 1987. doi:10.1145/22719.22720.

Corchuelo, Rafael, José A. Pérez, Antonio Ruiz, and Miguel Toro. *Repairing Syntax Errors in LR Parsers. ACM Transactions on Programming Languages and Systems* 24(6):698–710, 2002. doi:10.1145/586088.586092.

Denis, Xavier, Jacques-Henri Jourdan, and Claude Marché. *Creusot: A Foundry for the Deductive Verification of Rust Programs.* In *Formal Methods and Software Engineering (ICFEM 2022)*, LNCS 13478. Springer, 2022. doi:10.1007/978-3-031-17244-1_6.

Diekmann, Lukas. *Editing Composed Languages.* PhD thesis, King's College London, 2019.

Diekmann, Lukas, and Laurence Tratt. *Don't Panic! Better, Fewer, Syntax Errors for LR Parsers.* In *34th European Conference on Object-Oriented Programming (ECOOP 2020)*, LIPIcs 166:6:1–6:32, 2020. doi:10.4230/LIPIcs.ECOOP.2020.6.

Egolf, Derek, Sam Lasser, and Kathleen Fisher. *Verbatim: A Verified Lexer Generator.* In *2021 IEEE Security and Privacy Workshops (SPW)*, 92–100, 2021. doi:10.1109/SPW53761.2021.00022.

Floyd, Robert W. *Syntactic Analysis and Operator Precedence. Journal of the ACM* 10(3):316–333, 1963. doi:10.1145/321172.321179.

Hammer, Matthew A., Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. *Adapton: Composable, Demand-Driven Incremental Computation.* In *PLDI 2014*, 156–166. ACM, 2014. doi:10.1145/2594291.2594324.

Haverbeke, Marijn. *Lezer: A Parser System for CodeMirror 6.* 2020. https://lezer.codemirror.net/.

Jourdan, Jacques-Henri, François Pottier, and Xavier Leroy. *Validating LR(1) Parsers.* In *Programming Languages and Systems (ESOP 2012)*, LNCS 7211:397–416. Springer, 2012. doi:10.1007/978-3-642-28869-2_20.

Kani Contributors. *Kani Rust Verifier.* GitHub, 2024. https://github.com/model-checking/kani.

Lasser, Sam, Chris Casinghino, Kathleen Fisher, and Cody Roux. *CoStar: A Verified ALL(*) Parser.* In *PLDI 2021*, 420–434. ACM, 2021. doi:10.1145/3453483.3454053.

Lawlor, Ciaran. *ciaran16/incremental-parser.* GitHub, 2017. https://github.com/ciaran16/incremental-parser.

Leroy, Xavier. *Formal Verification of a Realistic Compiler. Communications of the ACM* 52(7):107–115, 2009. doi:10.1145/1538788.1538814.

Li, Le, and Kenjiro Taura. *Associative Operator Precedence Parsing: A Method to Increase Data Parsing Parallelism.* In *HPC Asia 2023*, 75–87. ACM, 2023. doi:10.1145/3578178.3578233.

Lippert, Eric. *Persistence, Façades, and Roslyn's Red-Green Trees.* 2012. https://ericlippert.com/2012/06/08/red-green-trees/.

Microsoft .NET Foundation. *The Roslyn C# Compiler — Blender.cs.* 2014. https://github.com/dotnet/roslyn/blob/main/src/Compilers/CSharp/Portable/Parser/Blender.cs.

Microsoft. *TypeScript Compiler — IncrementalParser.* 2014. https://github.com/microsoft/TypeScript/tree/main/src/compiler.

Pratt, Vaughan R. *Top Down Operator Precedence.* In *Proceedings of the 1st Annual ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '73)*, 41–51. ACM, 1973. doi:10.1145/512927.512931.

Swift Project. *SwiftSyntax.* 2018. https://github.com/swiftlang/swift-syntax.

Wagner, Tim A., and Susan L. Graham. *Efficient and Flexible Incremental Parsing. ACM Transactions on Programming Languages and Systems* 20(5):980–1013, 1998. doi:10.1145/293677.293678.

Why3 Contributors. *Why3 — Where Programs Meet Provers.* 2024. https://www.why3.org/.